ViandasSemana

Módulo 5 — Plataforma

Validación, errores, registro de eventos, tests y CI/CD

Segundo Enriquez Gatti · Roman Oberlander
6.° 1.ª — E.T. N.° 21 — Programación sobre Redes — 2026

Módulo 5 — Plataforma y registro de eventos

Este módulo reúne lo que usan todos los demás: cómo se validan los datos y se responden los errores, cómo el sistema deja registro de lo que pasa (observabilidad), la configuración, los tests y el pipeline que publica cada versión. Es el **proceso 5 "Registrar eventos"** de los DFD.

No tiene casos de uso propios: es transversal y da soporte a los CU-01 a CU-17.

Errores claros

Toda entrada se valida al principio de cada ruta. Si algo no corresponde, el usuario recibe un mensaje claro (por ejemplo "Sólo quedan 3 viandas") y nunca un error técnico. Ningún dato inválido llega a la base de datos.

Registro de eventos

El servidor anota cada petición y cada hecho importante del negocio: un login fallido, una reserva rechazada por cupo u horario, un pago aprobado o devuelto, una falla de Mercado Pago o del correo. Esos registros se consultan en **Render Logs** y sirven, por ejemplo, para responder a un cliente que dice "no pude pedir": se busca la hora y se ve si fue por cupo, por horario o por una falla.

Nunca se registran contraseñas, tokens ni cookies.

Calidad y despliegue

El backend tiene **78 tests automáticos** que corren en cada cambio. Si falla alguno o la cobertura baja del **65 %**, la versión nueva no se publica y producción sigue como estaba.

Funcionamiento técnico

1. Arquitectura y stack tecnológico

Archivo	Qué hace
backend/src/server.ts	Punto de entrada: lee la configuración y levanta el servidor.
backend/src/app.ts	Arma la app Express: helmet, CORS, JSON de hasta 100 kB, logs HTTP, rutas y manejo de errores.
backend/src/config/env.ts	Única lectura de variables de entorno; falla al arrancar si falta DATABASE_URL o JWT_SECRET.
backend/src/lib/errors.ts	HttpError(status, mensaje, codigo?).
backend/src/middleware/errorHandler.ts	noEncontrado (404) y manejadorErrores (400 / 4xx / 500).
backend/src/lib/logger.ts	Logger pino en JSON, con datos sensibles ocultos.
backend/src/routes/health.ts	GET /api/health.
frontend/src/api/cliente.ts	Única puerta del frontend a la API (manejo de 401).
backend/vitest.config.ts	Tests, base de pruebas y umbral de cobertura del 65 %.
.github/workflows/ci-cd.yml	Pipeline de CI/CD.
docker-compose.yml	Bases locales de desarrollo (5432) y de tests (5433).

2. Validación y manejo de errores

Cada ruta valida con **zod** (`esquema.parse(req.body)`). Los errores viajan solos hasta el manejador central (Express 5 lo hace también en rutas `async`):

Situación	Respuesta
Datos que no cumplen el esquema (ZodError)	400 { error: "Datos inválidos", detalles: [{ campo, mensaje }] }
JSON mal formado	400 { error: "El cuerpo de la petición no es un JSON válido" }
Regla de negocio, permiso o recurso (HttpError)	su código (401, 403, 404, 409, 502, 503) con { error, codigo? }
Ruta inexistente	404 { error: "Recurso no encontrado", ruta }
Cualquier otro error	500 { error: "Error interno del servidor" }; el detalle va sólo al log

3. Observabilidad (proceso 5 "Registrar eventos")

La API escribe **logs estructurados en JSON** con pino en la salida estándar; en producción los guarda y muestra **Render Logs**. Cada línea tiene la hora, el nivel, `servicio: "viandassemana-api"` y, en los eventos de negocio, un campo `evento` por el que se puede buscar. `pino-http` agrega una línea por petición con método, ruta, código y tiempo.

evento	Nivel	Cuándo
login_fallido	warn	Email o contraseña incorrectos
recuperacion_solicitada	info	Se mandó un código de recuperación
password_restablecida	info	Se cambió una contraseña con código

evento	Nivel	Cuándo
regla_negocio	warn	Rechazo con código: CUPO_COMPLETO, FUERA_DE_HORARIO, YA_CANCELADO, SIN_PRECIO, NO_PAGABLE, CODIGO_*
pago_aprobado	info	Reserva pagada y confirmada
pago_devuelto	info / warn	Devolución por cancelación o por pago no esperado
mercadopago_sin_configurar / _sin_conexion / _error	error	Falla de Mercado Pago
correo_sin_configurar / correo_fallido	error	Falla del correo
error_servidor	error	Error no controlado (500)

Qué nunca se registra: el encabezado Authorization, las cookies, las contraseñas ni los hashes (`redact` en lib/logger.ts).

4. Salud y seguridad de la API

- `GET /api/health` → `200 { estado: "ok", servicio, hora }`. Lo usa Render para saber si la API está viva.
- `helmet` agrega encabezados de seguridad HTTP.
- CORS sólo acepta los orígenes de `CORS_ORIGIN` (en producción, la URL del frontend).
- El cuerpo JSON tiene un límite de 100 kB.
- Ningún secreto está en el código: todos salen de `config/env.ts`.

5. Configuración (variables de entorno)

Variable	Obligatoria	Uso
DATABASE_URL	sí	Conexión a PostgreSQL
JWT_SECRET	sí	Firma de tokens y HMAC de códigos de recuperación
PORT, NODE_ENV, CORS_ORIGIN, LOG_LEVEL	no	Puerto, entorno, orígenes permitidos, nivel de log
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, MAIL_FROM	no (sin ellas, recuperar da 503)	Correo
MP_ACCESS_TOKEN, FRONTEND_URL, API_URL	no (sin token, reservar da 503)	Mercado Pago
VITE_API_URL (frontend)	no	URL de la API, se fija al construir

6. Tests

`npm test` (en backend/) corre **78 tests**: 20 unitarios y 58 de integración, y genera la cobertura.

Archivo	Tipo	Tests
tests/unit/reglasPedido.test.ts	unitario	19
tests/unit/mailer.test.ts	unitario	1

Archivo	Tipo	Tests
tests/integration/auth.test.ts	integración	11
tests/integration/recuperacion.test.ts	integración	7
tests/integration/menus.test.ts	integración	10
tests/integration/pedidos.test.ts	integración	11
tests/integration/pagos.test.ts	integración	14
tests/integration/health.test.ts	integración	5

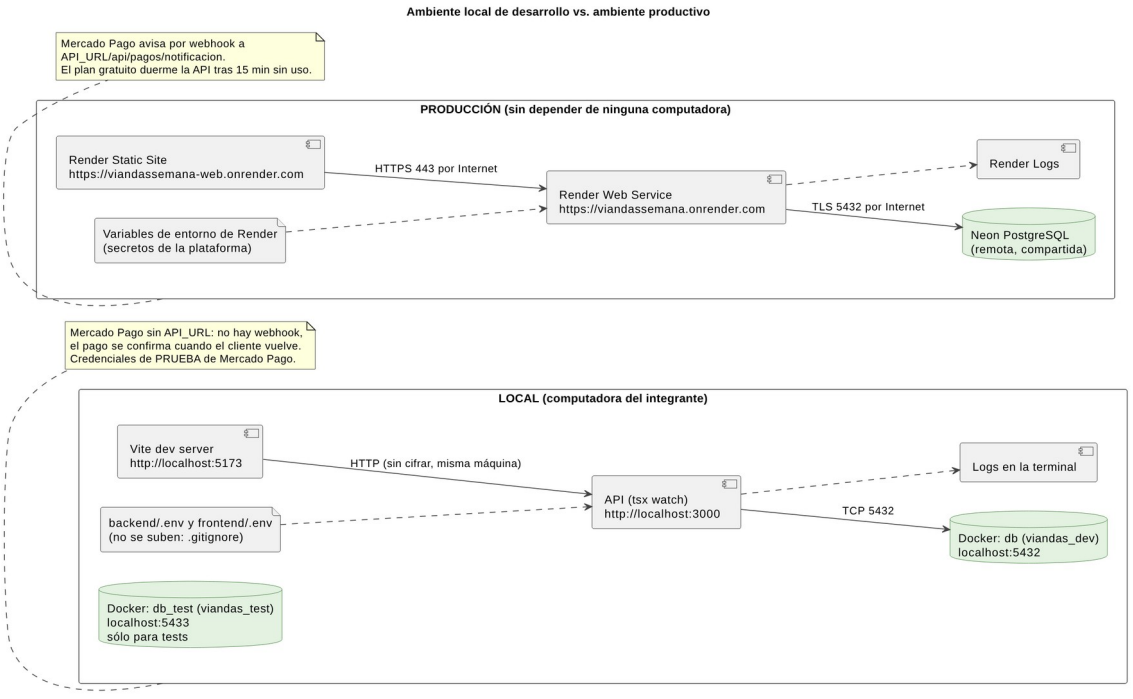
- Los de integración hacen la petición HTTP completa con supertest contra una **base de pruebas separada** (localhost:5433/viandas_test), nunca la de desarrollo ni la de producción.
- Mercado Pago se simula reemplazando fetch, y el correo con vi.mock.
- **Cobertura:** si las líneas cubiertas bajan del 65 %, npm test falla. Sólo se excluyen src/server.ts y src/config/**.

7. Pipeline de CI/CD

Se dispara con cada push y cada pull request a main (rama protegida: los cambios entran por pull request).

Job	Pasos	Cuándo corre
backend	npm ci → prisma generate → prisma migrate deploy sobre una base PostgreSQL efímera → npm test (umbral 65 %) → sube la cobertura → npm run build	push y PR
frontend	npm ci → npm run build	push y PR
desplegar	curl a los deploy hooks de Render del backend y del frontend	sólo en push a main y si backend y frontend pasaron

Si falla un test o baja la cobertura, `desplegar` no corre y producción no cambia.



Ambiente local y de producción (archivo editable: diagramas/ambientes.drawio).